QADRI GROUP | QADBROS

Supply Chain Automation

Project Progress Report

Prepared: 18 July 2026

Repository branch: master | Database: supply_chain_db (PostgreSQL)

3	21	~104k	AI
Domains live	DB tables/views	Rows loaded	Text-to-SQL

This report summarizes the current state of the QADBROS supply-chain automation platform: a PostgreSQL data warehouse fed by an Excel-to-DB ETL pipeline across three business domains (Logistics/Exports, Imports, and Stores), an AI text-to-SQL assistant, and a Streamlit web dashboard. It reflects both committed work and the current uncommitted working tree, and flags the open risks that need attention before the next milestone.

1. Executive Summary

The platform has moved from an imports-only prototype to a full three-domain data warehouse. The database is live and populated with real operational data, and the AI assistant answers natural-language questions directly against it. The web dashboard exists as a working prototype but its business pages still run on mock data and are not yet wired to the database.

Where things stand at a glance

Database schema (all domains)	DONE	21 tables + views created and populated.
ETL pipeline (Logistics)	DONE	load_01-05, tested and loading.
ETL pipeline (Imports)	DONE	load_06-10, 451 imports loaded.
ETL pipeline (Stores)	DONE	items, stock, issuance, purchases, AB items.
AI text-to-SQL assistant	DONE	OpenAI gpt-4o-mini, rich business rules, live.
Streamlit app shell	IN PROGRESS	Runs; pages still on mock data.
Backend data-access layer	NEEDS FIX	Facade added but imports a missing 'stubs' module.
Security / secrets hygiene	NEEDS FIX	Live API key + DB password in tracked/plaintext files.
Version control state	IN PROGRESS	Large body of work uncommitted since last commit.

Top priorities

- **1.** Remove the missing-'stubs' dependency so the backend facade actually imports.
- **2.** Rotate the exposed OpenAI key and DB password, then move all secrets to .env only.
- **3.** Wire the Streamlit pages to backend.data_access (real DB) instead of utils.mock_data.
- **4.** Commit the current working tree and stop tracking __pycache__ / .pyc files.

2. System Architecture

The system is organized in clear layers. Source Excel workbooks are cleaned and loaded by the ETL scripts into PostgreSQL. Two consumers sit on top of the database: the AI assistant (natural language -> SQL) and the Streamlit dashboard.

Layered flow

Source data	Excel / xls workbooks in data/ (logistics, imports, stores, ab_items ...)
ETL layer	database/scripts/* - clean + load via shared etl_common helpers
Database	PostgreSQL supply_chain_db - 21 tables & views, ~104k rows
AI assistant	chatbot/agent.py - LangGraph text-to-SQL over the live schema
Web app	app.py + pages/* - Streamlit dashboard (prototype)
Backend facade	backend/data_access.py - single data API for pages (in progress)

Repository map (source, excluding venv & data)

Path	Purpose
app.py, pages/	Streamlit UI: dashboard, purchase, inventory, imports, reports, chatbot
backend/	data_access facade + db_connection (SQLAlchemy engine)
chatbot/	agent.py (text-to-SQL), config.py (API key/model)
database/connection/	psycopg2 connection used by ETL & schema scripts
database/schemas/	CREATE TABLE definitions: logistics, imports, stores + views
database/scripts/	load_all orchestrator + per-sheet loaders + etl_common helpers
utils/	mock_data.py (placeholder data), styles.py (UI styling)
data/	Source workbooks + exported CSVs (git-ignored)

3. Database - Live Data

The database is live and fully populated across all three domains. Row counts below are from a direct query of supply_chain_db at report time - concrete evidence that the full pipeline runs end to end, not just imports as in earlier milestones.

Imports module

Table	Rows	Notes
import_details	451	One row per real import (surrogate import_id)
import_item	161	Line items (rows carrying an item code)
shipment_details	451	Per batch / B-L shipment records
payment_history	451	Per payment event

Logistics / Exports module

Table	Rows	Notes
exports	163	Export master records
export_documents	3,053	Documentation per export
export_shipments	165	Shipment sheet
shipment_containers	80	20ft / 40ft container lines
packing_details	1,375	Packing sheet
shifting_movements	464	Inbound / outbound movements
v_* views	4	Documentation / packing / shifting / shipment metrics

Stores + shared masters

Table	Rows	Notes
items	26,818	Shared master - item_code natural key
purchase_order	802	Shared master
purchases_data	2,778	Purchases list
issuance	50,711	Consumption transactions (largest table)
stock	7,748	Current stock snapshot
store_requisition	7,075	Store requisitions
ab_items	304	A/B classification: rank, safety & lead-time days

Load orchestration: database/scripts/load_all.py performs a clean reload - it drops and recreates all schemas, points each loader at the real workbook under data/, truncates transaction tables, then repopulates. Masters (items / purchase_order) are upserted idempotently. Run with: python -m database.scripts.load_all

4. ETL Pipeline

Loaders share a common set of cleaning helpers (etl_common: read_sheet, clean_text, clean_date, clean_int, etc.) so every sheet is normalized the same way. Each loader targets one source sheet and writes to one or more tables.

Loader inventory

Script	Domain	Loads
logistics/load_01_exports	Logistics	exports
logistics/load_02_export_documentation	Logistics	export_documents
logistics/load_03_shipments	Logistics	export_shipments, containers
logistics/load_04_packing	Logistics	packing_details
logistics/load_05_shifting	Logistics	shifting_movements
stores/load_01_items	Stores	items (master)
stores/load_02_purchases_data	Stores	purchases_data
stores/load_03_purchase_order	Stores	purchase_order (master)
stores/load_04_issuance	Stores	issuance
stores/load_05_stock	Stores	stock
stores/load_06_store_requisitions	Stores	store_requisition
stores/load_07_ab_items	Stores	ab_items (new)
load_06_import_masters	Imports	items/suppliers/PO from imports
load_07_import_details	Imports	import_details
load_08_import_items	Imports	import_item
load_09_shipment_details	Imports	shipment_details
load_10_payment_history	Imports	payment_history

Imports load model (design note)

- The Import Status Sheet has no natural key and PO No is empty, so import_id is a DB-generated surrogate (BIGSERIAL).
- One import per real data row (~451 of ~1,727 rows; the rest are trailing formula-only rows).
- Child loaders link to their parent via a load-time R<row-index> reference, not business data.

5. AI Assistant - Text-to-SQL

chatbot/agent.py is a LangGraph agent that turns a natural-language question into a single read-only PostgreSQL SELECT, runs it, and summarizes the result. It is wired into the Streamlit app as the 'Supply Chain Assistant' page, returning text, an interactive table (with CSV download), or a chart depending on the question.

How it works

- **Flow:** generate_sql -> execute_sql -> (retry on error, up to 3 attempts) -> summarize.
- **Schema-aware:** Reads the live schema (CREATE TABLE + sample rows) so prompts always match the real database.
- **Safety:** is_safe_select blocks any write/DDL; only SELECT/WITH allowed; results auto-capped at 1,000 rows.
- **Output:** detect_display picks text vs table vs chart from the wording of the question.

Notable change since last milestone

The assistant was migrated from Google Gemini to OpenAI (gpt-4o-mini via langchain-openai). The system prompt now encodes extensive domain rules: imports vs exports must never be joined; import status vocabulary; overdue / upcoming ETA logic; correct handling of the three purchase dates (demand vs required-by vs purchased); per-day vs average-delay math; NULL-safe ranking; company aliases (QCL, QBL2, QEN, QE); and always attaching UOM to quantities. This is a substantial accuracy upgrade over a generic prompt.

6. Web App & Backend Layer

Streamlit pages

Page	Data source	State
app.py (home)	Hardcoded metrics	Prototype
dashboard.py	utils.mock_data	Mock data
purchase.py	utils.mock_data	Mock data
inventory.py	utils.mock_data	Mock data
imports.py	Hardcoded metrics	Mock data
reports.py	None (UI only)	Stub
chatbot.py	chatbot.agent (LIVE DB)	Working on real data

Only the chatbot page is on real data today. The other pages read placeholder frames from utils/mock_data.py.

Backend facade (backend/data_access.py)

A facade layer was introduced as the single data API for pages: dashboard KPIs, purchases, stock, imports, logistics, plus executive/enriched functions (supplier performance, aging, health, status splits). The intent is to swap only the function bodies from stubs to real queries later, without changing any page code.

- **Live:** stock() is already implemented against the real database (joins stock -> items, derives a status).
- **Blocking:** Every other function forwards to 'from stubs import fake_data' - but there is NO stubs/ directory in the repo, so importing data_access currently fails.
- **Not wired:** The pages do not import the facade yet; they still use utils.mock_data directly.

7. Open Issues & Risks

Security - needs immediate action

- **Key exposure:** A live OpenAI API key is stored in plaintext in chatbot/config.py. It is git-ignored (not committed) but still sits on disk and was pasted into source - it should be rotated and moved to .env only.
- **DB password:** The PostgreSQL password (345waleed) is hardcoded in database/connection/database_connection.py. The .gitignore rule '/connection/database_connection.py' is anchored to the repo root and does NOT match the real path (database/connection/...), so this file - and the password - IS tracked in git.

Correctness / structure

- **Broken import:** backend/data_access.py imports a non-existent 'stubs' module - the facade cannot be imported as-is.
- **Config drift:** Two different default DB passwords exist (backend/db_connection.py uses '0000'; the psycopg2 connection uses '345waleed'). Consolidate on one .env-driven config.
- **Stray file:** database/scripts/read.py is a scratch file with a hardcoded C:/Users/hp/... path - remove it.

Hygiene

- **Cache noise:** __pycache__ and .pyc files are tracked and churn between Python 3.12 and 3.14 builds, creating noisy diffs. Add them to .gitignore and untrack.
- **Uncommitted:** A large body of work (backend layer, edits across nearly all load scripts, chatbot changes) is uncommitted. Commit in logical chunks to checkpoint progress.

8. Recommended Next Steps

#	Action	Priority
1	Rotate exposed OpenAI key + DB password; move all secrets to .env	Critical
2	Fix .gitignore so database_connection.py is not tracked	Critical
3	Remove/replace the missing 'stubs' import in data_access.py	High
4	Wire Streamlit pages to backend.data_access (real DB queries)	High
5	Untrack __pycache__/*.pyc and add to .gitignore	Medium
6	Commit current working tree in logical chunks	Medium
7	Replace placeholder 'Below reorder' rule with a real reorder threshold	Low
8	Delete scratch file database/scripts/read.py	Low

Report generated from a direct review of the repository source and a live query of supply_chain_db. Row counts and file states reflect the working tree at the time of writing.